```
/**
 * Rehashing for quadratic probing hash table.
 */
void rehash( )
{
    vector<HashEntry> oldArray = array;

        // Create new double-sized, empty table
    array.resize( nextPrime( 2 * oldArray.size( ) ) );
    for( auto & entry : array )
        entry.info = EMPTY;

        // Copy table over
    currentSize = 0;
    for( auto & entry : oldArray )
        if( entry.info == ACTIVE )
            insert( std::move( entry.element ) );
}

/**
 * Rehashing for separate chaining hash table.
 */
void rehash( )
{
    vector<list<HashedObj>> oldLists = theLists;

        // Create new double-sized, empty table
    theLists.resize( nextPrime( 2 * theLists.size( ) ) );
    for( auto & thisList : theLists )
        thisList.clear( );

        // Copy table over
    currentSize = 0;
    for( auto & thisList : oldLists )
        for( auto & x : thisList )
            insert( std::move( x ) );
}
```

Figure 5.22 Rehashing for both separate chaining hash tables and probing hash tables

also be instantiated with a function object that provides (or overrides a default) comparison function, `unordered_set` and `unordered_map` can be instantiated with function objects that provide a hash function and equality operator. Thus, for example, Figure 5.23 illustrates how an unordered set of *case-insensitive* strings can be maintained, assuming that some string operations are implemented elsewhere.

```
class CaseInsensitiveStringHash
{
  public:
    size_t operator( ) ( const string & s ) const
    {
        static hash<string> hf;
        return hf( toLower( s ) );            // toLower implemented elsewhere
    }

    bool operator( ) ( const string & lhs, const string & rhs ) const
    {
        return equalsIgnoreCase( lhs, rhs );  // equalsIgnoreCase is elsewhere
    }
};

unordered_set<string,CaseInsensitiveStringHash,CaseInsensitiveStringHash> s;
```

Figure 5.23 Creating a case-insensitive `unordered_set`

These unordered classes can be used if it is not important for the entries to be viewable in sorted order. For instance, in the word-changing example in Section 4.8, there were three maps:

1. A map in which the key is a *word length*, and the value is a collection of all words of that word length.
2. A map in which the key is a *representative*, and the value is a collection of all words with that representative.
3. A map in which the key is a *word*, and the value is a collection of all words that differ in only one character from that word.

Because the order in which word lengths are processed does not matter, the first map can be an `unordered_map`. Because the representatives are not even needed after the second map is built, the second map can be an `unordered_map`. The third map can also be an `unordered_map`, unless we want `printHighChangeables` to alphabetically list the subset of words that can be changed into a large number of other words.

The performance of an `unordered_map` can often be superior to a `map`, but it is hard to know for sure without writing the code both ways.

5.7 Hash Tables with Worst-Case $O(1)$ Access

The hash tables that we have examined so far all have the property that with reasonable load factors, and appropriate hash functions, we can expect $O(1)$ cost on average for insertions, removes, and searching. But what is the expected worst case for a search assuming a reasonably well-behaved hash function?

For separate chaining, assuming a load factor of 1, this is one version of the classic **balls and bins problem**: Given N balls placed randomly (uniformly) in N bins, what is the expected number of balls in the most occupied bin? The answer is well known to be $\Theta(\log N/\log\log N)$, meaning that on average, we expect some queries to take nearly logarithmic time. Similar types of bounds are observed (or provable) for the length of the longest expected probe sequence in a probing hash table.

We would like to obtain $O(1)$ worst-case cost. In some applications, such as hardware implementations of lookup tables for routers and memory caches, it is especially important that the search have a definite (i.e., constant) amount of completion time. Let us assume that N is known in advance, so no rehashing is needed. If we are allowed to rearrange items as they are inserted, then $O(1)$ worst-case cost is achievable for searches.

In the remainder of this section we describe the earliest solution to this problem, namely perfect hashing, and then two more recent approaches that appear to offer promising alternatives to the classic hashing schemes that have been prevalent for many years.

5.7.1 Perfect Hashing

Suppose, for purposes of simplification, that all N items are known in advance. If a separate chaining implementation could guarantee that each list had at most a constant number of items, we would be done. We know that as we make more lists, the lists will on average be shorter, so theoretically if we have enough lists, then with a reasonably high probability we might expect to have no collisions at all!

But there are two fundamental problems with this approach: First, the number of lists might be unreasonably large; second, even with lots of lists, we might still get unlucky.

The second problem is relatively easy to address in principle. Suppose we choose the number of lists to be M (i.e., *TableSize* is M), which is sufficiently large to guarantee that with probability at least $\frac{1}{2}$, there will be no collisions. Then if a collision is detected, we simply clear out the table and try again using a different hash function that is independent of the first. If we still get a collision, we try a third hash function, and so on. The expected number of trials will be at most 2 (since the success probability is $\frac{1}{2}$), and this is all folded into the insertion cost. Section 5.8 discusses the crucial issue of how to produce additional hash functions.

So we are left with determining how large M, the number of lists, needs to be. Unfortunately, M needs to be quite large; specifically $M = \Omega(N^2)$. However, if $M = N^2$, we can show that the table is collision free with probability at least $\frac{1}{2}$, and this result can be used to make a workable modification to our basic approach.

Theorem 5.2

If N balls are placed into $M = N^2$ bins, the probability that no bin has more than one ball is less than $\frac{1}{2}$.

Proof

If a pair (i, j) of balls are placed in the same bin, we call that a collision. Let $C_{i,j}$ be the expected number of collisions produced by any two balls (i, j). Clearly the probability that any two specified balls collide is $1/M$, and thus $C_{i,j}$ is $1/M$, since the number of collisions that involve the pair (i, j) is either 0 or 1. Thus the expected number of

collisions in the entire table is $\sum_{(i,j),\ i<j} C_{i,j}$. Since there are $N(N-1)/2$ pairs, this sum is $N(N-1)/(2M) = N(N-1)/(2N^2) < \frac{1}{2}$. Since the expected number of collisions is below $\frac{1}{2}$, the probability that there is even one collision must also be below $\frac{1}{2}$.

Of course, using N^2 lists is impractical. However, the preceding analysis suggests the following alternative: Use only N bins, but resolve the collisions in each bin by using hash tables instead of linked lists. The idea is that because the bins are expected to have only a few items each, the hash table that is used for each bin can be quadratic in the bin size. Figure 5.24 shows the basic structure. Here, the primary hash table has ten bins. Bins 1, 3, 5, and 7 are all empty. Bins 0, 4, and 8 have one item, so they are resolved by a secondary hash table with one position. Bins 2 and 6 have two items, so they will be resolved into a secondary hash table with four (2^2) positions. And bin 9 has three items, so it is resolved into a secondary hash table with nine (3^2) positions.

As with the original idea, each secondary hash table will be constructed using a different hash function until it is collision free. The primary hash table can also be constructed several times if the number of collisions that are produced is higher than required. This scheme is known as **perfect hashing**. All that remains to be shown is that the total size of the secondary hash tables is indeed expected to be linear.

Theorem 5.3

If N items are placed into a primary hash table containing N bins, then the total size of the secondary hash tables has expected value at most $2N$.

Proof

Using the same logic as in the proof of Theorem 5.2, the expected number of pairwise collisions is at most $N(N-1)/2N$, or $(N-1)/2$. Let b_i be the number of items that hash to position i in the primary hash table; observe that b_i^2 space is used for this cell

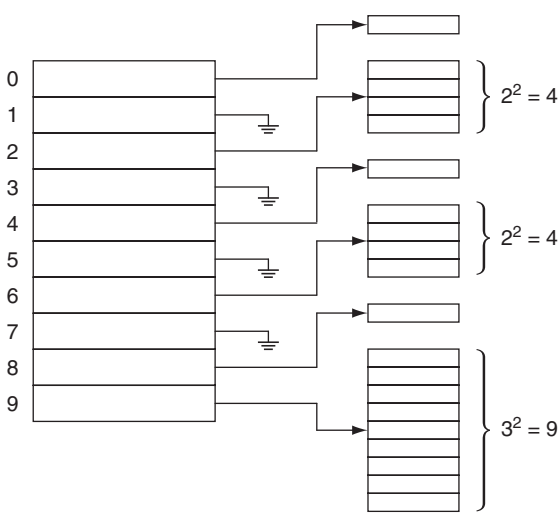

Figure 5.24 Perfect hashing table using secondary hash tables

in the secondary hash table, and that this accounts for $b_i\,(b_i - 1)/2$ pairwise collisions, which we will call c_i. Thus the amount of space, b_i^2, used for the ith secondary hash table is $2c_i + b_i$. The total space is then $2\sum c_i + \sum b_i$. The total number of collisions is $(N-1)/2$ (from the first sentence of this proof); the total number of items is of course N, so we obtain a total secondary space requirement of $2(N-1)/2 + N < 2N$.

Thus, the probability that the total secondary space requirement is more than $4N$ is at most $\frac{1}{2}$ (since, otherwise, the expected value would be higher than $2N$), so we can keep choosing hash functions for the primary table until we generate the appropriate secondary space requirement. Once that is done, each secondary hash table will itself require only an average of two trials to be collision free. After the tables are built, any lookup can be done in two probes.

Perfect hashing works if the items are all known in advance. There are dynamic schemes that allow insertions and deletions (**dynamic perfect hashing**), but instead we will investigate two newer alternatives that appear to be competitive in practice with the classic hashing algorithms.

5.7.2 Cuckoo Hashing

From our previous discussion, we know that in the balls and bins problem, if N items are randomly tossed into N bins, the size of the largest bin is expected to be $\Theta(\log N/\log\log N)$. Since this bound has been known for a long time, and the problem has been well studied by mathematicians, it was surprising when, in the mid 1990s, it was shown that if, at each toss, two bins were randomly chosen and the item was tossed into the more empty bin (at the time), then the size of the largest bin would only be $\Theta(\log\log N)$, a significantly lower number. Quickly, a host of potential algorithms and data structures arose out of this new concept of the "power of two choices."

One of the ideas is **cuckoo hashing**. In cuckoo hashing, suppose we have N items. We maintain two tables, each more than half empty, and we have two independent hash functions that can assign each item to a position in each table. Cuckoo hashing maintains the invariant that an item is always stored in one of these two locations.

As an example, Figure 5.25 shows a potential cuckoo hash table for six items, with two tables of size 5 (these tables are too small, but serve well as an example). Based on the

Table 1	
0	B
1	C
2	
3	E
4	

Table 2	
0	D
1	
2	A
3	
4	F

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Figure 5.25 Potential cuckoo hash table. Hash functions are shown on the right. For these six items, there are only three valid positions in Table 1 and three valid positions in Table 2, so it is not clear that this arrangement can easily be found.

randomly chosen hash functions, item *A* can be at either position 0 in Table 1 or position 2 in Table 2. Item *F* can be at either position 3 in Table 1 or position 4 in Table 2, and so on. Immediately, this implies that a search in a cuckoo hash table requires at most two table accesses, and a remove is trivial, once the item is located (lazy deletion is not needed now!).

But there is an important detail: How is the table built? For instance, in Figure 5.25, there are only three available locations in the first table for the six items, and there are only three available locations in the second table for the six items. So there are only six available locations for these six items, and thus we must find an ideal matching of slots for our six items. Clearly, if there were a seventh item, *G*, with locations 1 for Table 1 and 2 for Table 2, it could not be inserted into the table by any algorithm (the seven items would be competing for six table locations). One could argue that this means that the table would simply be too loaded (*G* would yield a 0.70 load factor), but at the same time, if the table had thousands of items, and were lightly loaded, but we had *A*, *B*, *C*, *D*, *E*, *F*, *G* with these hash positions, it would still be impossible to insert all seven of those items. So it is not at all obvious that this scheme can be made to work. The answer in this situation would be to pick another hash function, and this can be fine as long as it is unlikely that this situation occurs.

The cuckoo hashing algorithm itself is simple: To insert a new item, *x*, first make sure it is not already there. We can then use the first hash function, and if the (first) table location is empty, the item can be placed. So Figure 5.26 shows the result of inserting *A* into an empty hash table.

Suppose now we want to insert *B*, which has hash locations 0 in Table 1 and 0 in Table 2. For the remainder of the algorithm, we will use (h_1, h_2) to specify the two locations, so *B*'s locations are given by (0, 0). Table 1 is already occupied in position 0. At this point there are two options: One is to look in Table 2. The problem is that position 0 in Table 2 could also be occupied. It happens that in this case it is not, but the algorithm that the standard cuckoo hash table uses does not bother to look. Instead, it preemptively places the new item *B* in Table 1. In order to do so, it must displace *A*, so *A* moves to Table 2, using its Table 2 hash location, which is position 2. The result is shown in Figure 5.27. It is easy to insert *C*, and this is shown in Figure 5.28.

Next we want to insert *D*, with hash locations (1, 0). But the Table 1 location (position 1) is already taken. Note also that the Table 2 location is not already taken, but we don't look there. Instead, we have *D* replace *C*, and then *C* goes into Table 2 at position 4, as suggested by its second hash function. The resulting tables are shown in Figure 5.29.

Table 1	
0	A
1	
2	
3	
4	

Table 2	
0	
1	
2	
3	
4	

A: 0, 2

Figure 5.26 Cuckoo hash table after insertion of *A*

Table 1	
0	B
1	
2	
3	
4	

Table 2	
0	
1	
2	A
3	
4	

A: 0, 2
B: 0, 0

Figure 5.27 Cuckoo hash table after insertion of *B*

Table 1	
0	B
1	C
2	
3	
4	

Table 2	
0	
1	
2	A
3	
4	

A: 0, 2
B: 0, 0
C: 1, 4

Figure 5.28 Cuckoo hash table after insertion of *C*

After this is done, *E* can be easily inserted. So far, so good, but can we now insert *F*? Figures 5.30 to 5.33 show that this algorithm successfully inserts *F*, by displacing *E*, then *A*, and then *B*.

Clearly, as we mentioned before, we cannot successfully insert *G* with hash locations (1, 2). If we were to try, we would displace *D*, then *B*, then *A*, *E*, *F*, and *C*, and then *C*

Table 1	
0	B
1	D
2	
3	E
4	

Table 2	
0	
1	
2	A
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2

Figure 5.29 Cuckoo hash table after insertion of *D*

Table 1	
0	B
1	D
2	
3	**F**
4	

Table 2	
0	
1	
2	A
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Figure 5.30 Cuckoo hash table starting the insertion of *F* into the table in Figure 5.29. First, *F* displaces *E*.

Table 1	
0	B
1	D
2	
3	**F**
4	

Table 2	
0	
1	
2	**E**
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Figure 5.31 Continuing the insertion of *F* into the table in Figure 5.29. Next, *E* displaces *A*.

Table 1	
0	**A**
1	D
2	
3	**F**
4	

Table 2	
0	
1	
2	**E**
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Figure 5.32 Continuing the insertion of *F* into the table in Figure 5.29. Next, *A* displaces *B*.

Table 1	
0	**A**
1	D
2	
3	**F**
4	

Table 2	
0	**B**
1	
2	**E**
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Figure 5.33 Completing the insertion of *F* into the table in Figure 5.29. Miraculously (?), *B* finds an empty position in Table 2.

would try to go back into Table 1, position 1, displacing *G* which was placed there at the start. This would get us to Figure 5.34. So now *G* would try its alternate in Table 2 (location 2) and then displace *A*, which would displace *B*, which would displace *D*, which would displace *C*, which would displace *F*, which would displace *E*, which would now displace *G* from position 2. At this point, *G* would be in a cycle.

The central issue then concerns questions such as what is the probability of there being a cycle that prevents the insertion from completing, and what is the expected number of displacements required for a successful insertion? Fortunately, if the table's load factor is below 0.5, an analysis shows that the probability of a cycle is very low, that the expected number of displacements is a small constant, and that it is extremely unlikely that a successful insertion would require more than $O(\log N)$ displacements. As such, we can simply rebuild the tables with new hash functions after a certain number of displacements are

Table 1	
0	B
1	C
2	
3	E
4	

Table 2	
0	D
1	
2	A
3	
4	F

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4
G: 1, 2

Figure 5.34 Inserting *G* into the table in Figure 5.33. *G* displaces *D*, which displaces *B*, which displaces *A*, which displaces *E*, which displaces *F*, which displaces *C*, which displaces *G*. It is not yet hopeless since when *G* is displaced, we would now try the other hash table, at position 2. However, while that could be successful in general, in this case there is a cycle and the insertion will not terminate.

detected. More precisely, the probability that a single insertion would require a new set of hash functions can be made to be $O(1/N^2)$; the new hash functions themselves generate N more insertions to rebuild the table, but even so, this means the rebuilding cost is minimal. However, if the table's load factor is at 0.5 or higher, then the probability of a cycle becomes drastically higher, and this scheme is unlikely to work well at all.

After the publication of cuckoo hashing, numerous extensions were proposed. For instance, instead of two tables, we can use a higher number of tables, such as 3 or 4. While this increases the cost of a lookup, it also drastically increases the theoretical space utilization. In some applications the lookups through separate hash functions can be done in parallel and thus cost little to no additional time. Another extension is to allow each table to store multiple keys; again, this can increase space utilization and make it easier to do insertions and can be more cache-friendly. Various combinations are possible, as shown in Figure 5.35. And finally, often cuckoo hash tables are implemented as one giant table with two (or more) hash functions that probe the entire table, and some variations attempt to place an item in the second hash table immediately if there is an available spot, rather than starting a sequence of displacements.

Cuckoo Hash Table Implementation

Implementing cuckoo hashing requires a collection of hash functions; simply using `hashCode` to generate the collection of hash functions makes no sense, since any `hashCode` collisions will result in collisions in all the hash functions. Figure 5.36 shows a simple interface that can be used to send families of hash functions to the cuckoo hash table.

	1 item per cell	2 items per cell	4 items per cell
2 hash functions	0.49	0.86	0.93
3 hash functions	0.91	0.97	0.98
4 hash functions	0.97	0.99	0.999

Figure 5.35 Maximum load factors for cuckoo hashing variations

```
template <typename AnyType>
class CuckooHashFamily
{
  public:
    size_t hash( const AnyType & x, int which ) const;
    int getNumberOfFunctions( );
    void generateNewFunctions( );
};
```

Figure 5.36 Generic `HashFamily` interface for cuckoo hashing

Figure 5.37 provides the class interface for cuckoo hashing. We will code a variant that will allow an arbitrary number of hash functions (specified by the `HashFamily` template parameter type) which uses a single array that is addressed by all the hash functions. Thus our implementation differs from the classic notion of two separately addressable hash tables. We can implement the classic version by making relatively minor changes to the code; however, the version provided in this section seems to perform better in tests using simple hash functions.

In Figure 5.37, we specify that the maximum load for the table is 0.4; if the load factor of the table is about to exceed this limit, an automatic table expansion is performed. We also define `ALLOWED_REHASHES`, which specifies how many rehashes we will perform if evictions take too long. In theory, `ALLOWED_REHASHES` can be infinite, since we expect only a small constant number of rehashes are needed; in practice, depending on several factors such as the number of hash functions, the quality of the hash functions, and the load factor, the rehashes could significantly slow things down, and it might be worthwhile to expand the table, even though this will cost space. The data representation for the cuckoo hash table is straightforward: We store a simple array, the current size, and the collections of hash functions, represented in a `HashFamily` instance. We also maintain the number of hash functions, even though that is always obtainable from the `HashFamily` instance.

Figure 5.38 shows the constructor and `makeEmpty` methods, and these are straightforward. Figure 5.39 shows a pair of private methods. The first, `myHash`, is used to select the appropriate hash function and then scale it into a valid array index. The second, `findPos`, consults all the hash functions to return the index containing item `x`, or -1 if `x` is not found. `findPos` is then used by `contains` and `remove` in Figures 5.40 and 5.41, respectively, and we can see that those methods are easy to implement.

The difficult routine is insertion. In Figure 5.42, we can see that the basic plan is to check to see if the item is already present, returning if so. Otherwise, we check to see if the table is fully loaded, and if so, we expand it. Finally we call a helper routine to do all the dirty work.

The helper routine for insertion is shown in Figure 5.43. We declare a variable `rehashes` to keep track of how many attempts have been made to rehash in this insertion. Our insertion routine is mutually recursive: If needed, `insert` eventually calls `rehash`, which eventually calls back into `insert`. Thus `rehashes` is declared in an outer scope for code simplicity.

```
template <typename AnyType, typename HashFamily>
class CuckooHashTable
{
  public:
    explicit CuckooHashTable( int size = 101 );

    void makeEmpty( );
    bool contains( const AnyType & x ) const;

    bool remove( const AnyType & x );
    bool insert( const AnyType & x );
    bool insert( AnyType && x );

  private:
    struct HashEntry
    {
        AnyType element;
        bool isActive;

        HashEntry( const AnyType & e = AnyType( ), bool a = false )
          : element{ e }, isActive{ a } { }
        HashEntry( AnyType && e, bool a = false )
          : element{ std::move( e ) }, isActive{ a } { }
    };

    bool insertHelper1( const AnyType & xx );
    bool insertHelper1( AnyType && xx );
    bool isActive( int currentPos ) const;

    size_t myhash( const AnyType & x, int which ) const;
    int findPos( const AnyType & x ) const;
    void expand( );
    void rehash( );
    void rehash( int newSize );

    static const double MAX_LOAD = 0.40;
    static const int ALLOWED_REHASHES = 5;

    vector<HashEntry> array;
    int currentSize;
    int numHashFunctions;
    int rehashes;
    UniformRandom r;
    HashFamily hashFunctions;
};
```

Figure 5.37 Class interface for cuckoo hashing

```
    explicit HashTable( int size = 101 ) : array( nextPrime( size ) )
    {
        numHashFunctions = hashFunctions.getNumberOfFunctions( );
        rehashes = 0;
        makeEmpty( );
    }

    void makeEmpty( )
    {
        currentSize = 0;
        for( auto & entry : array )
            entry.isActive = false;
    }
```

Figure 5.38 Routines to initialize and empty the cuckoo hash table

```
    /**
     * Compute the hash code for x using specified function.
     */
    int myhash( const AnyType & x, int which ) const
    {
        return hashFunctions.hash( x, which ) % array.size( );
    }

    /**
     * Search all hash function places. Return the position
     * where the search terminates or -1 if not found.
     */
    int findPos( const AnyType & x ) const
    {
        for( int i = 0; i < numHashFunctions; ++i )
        {
            int pos = myhash( x, i );

            if( isActive( pos ) && array[ pos ].element == x )
                return pos;
        }

        return -1;
    }
```

Figure 5.39 Routines to find the location of an item in the cuckoo hash table and to compute the hash code for a given table

```
/**
 * Return true if x is found.
 */
bool contains( const AnyType & x ) const
{
    return findPos( x ) != -1;
}
```

Figure 5.40 Routine to search a cuckoo hash table

```
/**
 * Remove x from the hash table.
 * Return true if item was found and removed.
 */
bool remove( const AnyType & x )
{
   int currentPos = findPos( x );
   if( !isActive( currentPos ) )
       return false;

   array[ currentPos ].isActive = false;
   --currentSize;
   return true;
}
```

Figure 5.41 Routine to remove from a cuckoo hash table

```
1   bool insert( const AnyType & x )
2   {
3       if( contains( x ) )
4           return false;
5
6       if( currentSize >= array.size( ) * MAX_LOAD )
7           expand( );
8
9       return insertHelper1( x );
10  }
```

Figure 5.42 Public insert routine for cuckoo hashing

Our basic logic is different from the classic scheme. We have already tested that the item to insert is not already present. At lines 15 to 25, we check to see if any of the valid positions are empty; if so, we place our item in the first available position and we are done. Otherwise, we evict one of the existing items. However, there are some tricky issues:

```
static const int ALLOWED_REHASHES = 5;

bool insertHelper1( const AnyType & xx )
{
    const int COUNT_LIMIT = 100;
    AnyType x = xx;

    while( true )
    {
        int lastPos = -1;
        int pos;

        for( int count = 0; count < COUNT_LIMIT; ++count )
        {
            for( int i = 0; i < numHashFunctions; ++i )
            {
                pos = myhash( x, i );

                if( !isActive( pos ) )
                {
                    array[ pos ] = std::move( HashEntry{ std::move( x ), true } );
                    ++currentSize;
                    return true;
                }
            }

            // None of the spots are available. Evict a random one
            int i = 0;
            do
            {
                pos = myhash( x, r.nextInt( numHashFunctions ) );
            } while( pos == lastPos && i++ < 5 );

            lastPos = pos;
            std::swap( x, array[ pos ].element );
        }

        if( ++rehashes > ALLOWED_REHASHES )
        {
            expand( );     // Make the table bigger
            rehashes = 0;  // Reset the # of rehashes
        }
        else
            rehash( );     // Same table size, new hash functions
    }
}
```

Figure 5.43 Insertion routine for cuckoo hashing uses a different algorithm that chooses the item to evict randomly, attempting not to re-evict the last item. The table will attempt to select new hash functions (rehash) if there are too many evictions and will expand if there are too many rehashes.

- Evicting the first item did not perform well in experiments.
- Evicting the last item did not perform well in experiments.
- Evicting the items in sequence (i.e., the first eviction uses hash function 0, the next uses hash function 1, etc.) did not perform well in experiments.
- Evicting the item purely randomly did not perform well in experiments: In particular, with only two hash functions, it tended to create cycles.

To alleviate the last problem, we maintain the last position that was evicted, and if our random item was the last evicted item, we select a new random item. This will loop forever if used with two hash functions, and both hash functions happen to probe to the same location, and that location was a prior eviction, so we limit the loop to five iterations (deliberately using an odd number).

The code for `expand` and `rehash` is shown in Figure 5.44. `expand` creates a larger array but keeps the same hash functions. The zero-parameter `rehash` leaves the array size unchanged but creates a new array that is populated with newly chosen hash functions.

```
void expand( )
{
    rehash( static_cast<int>( array.size( ) / MAX_LOAD ) );
}

void rehash( )
{
    hashFunctions.generateNewFunctions( );
    rehash( array.size( ) );
}

void rehash( int newSize )
{
    vector<HashEntry> oldArray = array;

        // Create new double-sized, empty table
    array.resize( nextPrime( newSize ) );
    for( auto & entry : array )
        entry.isActive = false;

        // Copy table over
    currentSize = 0;
    for( auto & entry : oldArray )
        if( entry.isActive )
            insert( std::move( entry.element ) );
}
```

Figure 5.44 Rehashing and expanding code for cuckoo hash tables